Ridesharing Backend

Complete Interview Preparation Guide

Spring Boot • Kafka • Redis • Microservices • System Design • Managerial

#	Section	Questions
1	Ridesharing Project — Deep Dive	15
2	Spring Boot & Microservices	15
3	Kafka — Deep Dive	12
4	Redis — Deep Dive	10
5	System Design	10
6	Managerial Round	12

Section 1 — Ridesharing Project Deep Dive

Q1. Explain the overall architecture of your ride-sharing project. Hard

The system is built as 7 microservices: API Gateway, User Service, Driver Service, Ride Service, Pricing Service, Payment Service, and Notification Service. The API Gateway handles JWT auth and routing. Services communicate asynchronously via Apache Kafka for events (ride lifecycle, payments) and synchronously via REST for real-time reads. Redis stores driver geo-locations and surge pricing. Each service has its own PostgreSQL database (database-per-service pattern). Everything is containerised with Docker Compose.

Q2. How does ride matching work in your system? Hard

When a rider requests a ride, the Ride Service saves the ride with REQUESTED status and stores a Redis key ride:request:{id} with a 60-second TTL. It then publishes a ride.requested Kafka event. The Driver Service consumes this event and calls Redis GEORADIUS on the drivers:locations GeoSet to find ONLINE drivers within 5km, sorted by distance. The first driver to hit POST /rides/{id}/accept wins — the service checks if the Redis key still exists (not expired/accepted), assigns the driver, deletes the key, and publishes ride.accepted.

```
// Find nearest drivers — O(log N)
Circle circle = new Circle(new Point(lon, lat), new Distance(5,
Metrics.KILOMETERS));
GeoResults results = geoOps.radius("drivers:locations", circle, args);
```

Q3. What is the Ride state machine? What are all the states? Medium

REQUESTED (ride created, searching for driver) → DRIVER_ASSIGNED (driver accepted) → STARTED (trip in progress) → COMPLETED (trip ended, payment triggered). There are also CANCELLED (before or after assignment, not during trip) and PAYMENT_PENDING (completed but payment not yet confirmed). The state machine prevents invalid transitions — e.g., you cannot cancel a STARTED ride, and COMPLETED rides cannot go back to any prior state.

Q4. Why did you use Kafka instead of calling Payment Service directly from Ride Service? Hard

Decoupling — Ride Service should not be blocked or fail if Payment Service is down. With Kafka, Ride Service publishes ride.completed and its job is done. Payment Service picks it up when ready. This also allows multiple consumers (Notification Service listens to the same event). Additionally, Kafka guarantees durability — if Payment Service crashes mid-processing, Kafka retains the message and it's redelivered on restart. Direct REST calls would lose the event.

Q5. How did you prevent double-charging a rider? Hard

Two layers of idempotency: (1) Database check — before creating a Stripe PaymentIntent, Payment Service checks if a Payment row already exists for that rideId. If yes, it skips processing. (2) Stripe idempotency key — passing the rideId as Stripe's idempotency key ensures Stripe itself deduplicates if the same request is retried. This handles the case where the Kafka message is redelivered after a partial failure.

Q6. What happens if no driver accepts a ride within 60 seconds? Medium

The Redis key ride:request:{id} is stored with a 60-second TTL. When it expires, the ride is effectively stale. In production you can use Redis keyspace notifications (CONFIG SET notify-keyspace-events Ex) to listen for key expiry events. A listener would then mark the ride CANCELLED in the DB and publish a ride.cancelled Kafka event, which triggers a notification to the rider saying no driver was available. Alternatively, a scheduled job polls for REQUESTED rides older than 60 seconds.

Q7. How does surge pricing work? Medium

The Pricing Service maintains demand (ride requests) and supply (online drivers) counters per geo-zone in Redis with 5-minute TTLs. A @Scheduled job runs every 30 seconds, computes demand/supply ratio per zone, and writes the surge multiplier (capped between 1x and 3x) to Redis with a 30-second TTL. When fare is estimated, the current surge multiplier for the pickup zone is fetched from Redis and applied: finalFare = (baseFare + perKm + perMin) * surgeMultiplier.

Q8. How does the API Gateway handle authentication? Medium

A custom GlobalFilter (JwtAuthFilter) runs on every request. It extracts the Bearer token from the Authorization header, validates it using JJWT with the shared secret, and on success injects X-User-Id and X-User-Role headers into the forwarded request. Downstream services trust these headers — they never re-validate JWT. Public endpoints like /register and /login are in a whitelist and bypass the filter. Rate limiting is implemented per user using Redis counters in the gateway.

Q9. What is the Saga pattern and did you use it? Hard

The Saga pattern manages distributed transactions across microservices using a sequence of local transactions, each publishing an event that triggers the next step. In our system, the payment flow is a choreography-based saga: ride.completed → Payment Service charges → payment.success/failed. If payment fails, payment.failed triggers a compensating action (notify rider to retry, mark ride PAYMENT_PENDING). We don't use a central orchestrator — each service reacts to events, which is the choreography style.

Q10. How would you scale the Driver Location Service? Hard

Redis Cluster shards the GeoSet across nodes by key hash slot. For global scale, you can shard by city/region (drivers:locations:mumbai, drivers:locations:delhi). Driver location updates come every 5 seconds — with 100k drivers that's 20k writes/sec, well within Redis's throughput. For read scaling, Redis replicas serve GEORADIUS queries. You can also add a write buffer using a Kafka topic for location updates that a consumer batch-writes to Redis, reducing direct Redis write pressure.

Q11. How do you handle a failed payment — what is the retry strategy? Medium

On payment failure: (1) Payment Service updates status to FAILED, publishes payment.failed to Kafka. (2) Notification Service notifies the rider. (3) Ride status moves to PAYMENT_PENDING. For automatic retry, an exponential backoff retry can be configured on the Kafka consumer (@RetryableTopic with backoff). For Stripe downtime, the Resilience4j circuit breaker opens after 50% failure rate, preventing further calls and queuing events for retry when Stripe recovers.

Q12. Why does each microservice have its own database? Medium

This is the Database-per-Service pattern. Benefits: services are independently deployable (no shared schema migrations), each can choose the best DB type (Redis for geo, PostgreSQL for transactions), one service's heavy queries don't impact others, and it enforces loose coupling. The tradeoff is that joins across services are impossible — you use API calls or events to aggregate data. For reporting, a separate read model or data warehouse aggregates across services.

Q13. What is the role of Eureka in your project? Easy

Eureka is a service registry. Each microservice registers itself on startup with its host, port, and health status. The API Gateway and inter-service REST calls use lb:// URIs (load-balanced) — Spring Cloud LoadBalancer queries Eureka for available instances and round-robins requests. If a service instance goes down, Eureka removes it after missed heartbeats, and no new traffic is routed to it. This enables zero-downtime rolling deploys and horizontal scaling.

Q14. How is distributed tracing implemented? Medium

Zipkin is integrated via Spring Cloud Sleuth (Micrometer Tracing in Boot 3). Every HTTP request and Kafka message is tagged with a traceId and spanId. When Ride Service publishes to Kafka, the trace context is propagated in Kafka headers. Payment Service picks it up and continues the same trace. In Zipkin UI you can see the full end-to-end journey of a ride request — from API Gateway through Ride Service → Kafka → Payment Service — with timing for each hop.

Q15. What monitoring/observability did you set up? Medium

Three pillars: (1) Metrics — Spring Actuator exposes /actuator/prometheus. Prometheus scrapes all services every 15s. Grafana dashboards show JVM memory, HTTP request rates, error rates, Kafka consumer lag, and Redis hit ratio. (2) Tracing — Zipkin for distributed traces across services. (3) Logging — structured JSON logs with traceId/spanId for correlation. Kafka UI (provectuslabs) shows topic offsets, consumer group lag, and message inspection.

Section 2 — Spring Boot & Microservices

Q1. What is the difference between @RestController and @Controller? Easy

@Controller is a stereotype for MVC controllers that return views (HTML). @RestController = @Controller + @ResponseBody, meaning every method returns data (JSON/XML) directly serialised into the HTTP response body — no view resolution. In a REST API you always use @RestController. @Controller is used in Spring MVC web apps with Thymeleaf/JSP templates.

Q2. Explain Spring Boot Auto-configuration. Medium

Spring Boot scans classpath for libraries and auto-configures beans based on what's present. For example, if spring-boot-starter-data-jpa and a JDBC driver are on classpath, Spring Boot auto-configures a DataSource, EntityManagerFactory, and TransactionManager. Auto-configuration classes are listed in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports. You can override any auto-config bean by defining your own @Bean of the same type. Use @ConditionalOnMissingBean to provide defaults.

Q3. What is the difference between @Transactional on class vs method? Hard

On a class, @Transactional applies to all public methods of that class with the same propagation settings. On a method, it applies only to that method and overrides class-level settings. Key gotcha: @Transactional only works on public methods called from outside the bean (Spring creates a proxy). Calling a @Transactional method from within the same class bypasses the proxy and no transaction is started — this is a very common bug in interviews.

Q4. What is Circuit Breaker pattern? How did you use Resilience4j? Hard

A circuit breaker wraps calls to an external service. It tracks failure rate over a sliding window. When failures exceed a threshold, it 'opens' and immediately rejects calls (fallback is invoked) without trying the failing service — preventing cascade failures. After a wait period it goes to 'half-open' and lets a few probe calls through. If they succeed, it closes. In our project, @CircuitBreaker(name='stripe', fallbackMethod='stripePaymentFallback') wraps Stripe calls. Config: failureRateThreshold=50, waitDurationInOpenState=30s.

Q5. Explain Spring Cloud Gateway vs Zuul. Medium

Spring Cloud Gateway is non-blocking, built on Spring WebFlux and Reactor Netty, making it suitable for high-concurrency use cases. Zuul 1 is blocking (Servlet-based), which limits its throughput. Gateway supports reactive filters, better WebSocket proxying, and integrates natively with Spring Security reactive. Zuul 2 added non-blocking but has less Spring ecosystem integration. For new projects, Spring Cloud Gateway is the recommended choice.

Q6. What is the difference between EAGER and LAZY loading in JPA? Medium

EAGER loading fetches associated entities immediately with the parent query (JOIN). LAZY loading fetches them only when accessed (proxy + additional query). @ManyToOne and @OneToOne default to EAGER; @OneToMany and @ManyToMany default to LAZY. LAZY is generally preferred for performance. The famous 'N+1 problem' happens with LAZY when you iterate a collection and trigger N

separate queries. Fix: use JOIN FETCH in JPQL, @EntityGraph, or @BatchSize.

Q7. What is Spring Cloud Config and why use it? Medium

Spring Cloud Config Server provides centralised external configuration for all microservices from a Git repository (or file system). Services fetch their config on startup via HTTP. Benefits: (1) one place to change a value across all services, (2) environment-specific configs (dev/staging/prod), (3) config can be refreshed at runtime using @RefreshScope + /actuator/refresh without restart, (4) config is version-controlled in Git with full audit history.

Q8. How does Spring Security JWT authentication work? Medium

On login, UserDetailsService loads the user, PasswordEncoder verifies the password, and a JWT is generated (signed with HS256/RS256). On subsequent requests, a OncePerRequestFilter extracts the token from the Authorization header, validates signature and expiry using the same secret, and sets a UsernamePasswordAuthenticationToken in SecurityContextHolder. Spring Security then allows or denies the request based on @PreAuthorize / HttpSecurity rules. In our project, JWT validation is done at the API Gateway level.

Q9. What is the difference between @Bean and @Component? Easy

@Component (and its specialisations @Service, @Repository, @Controller) is used on classes you own — Spring auto-detects them via component scanning. @Bean is used in @Configuration classes to declare beans for third-party classes you can't annotate, or when you need programmatic control over instantiation (e.g., RedisTemplate with custom serialisers). Both register beans in the ApplicationContext — the difference is where the bean definition lives.

Q10. How do you handle exceptions globally in Spring Boot? Medium

@RestControllerAdvice (= @ControllerAdvice + @ResponseBody) combined with @ExceptionHandler methods provides centralised exception handling. You define methods like handleNotFoundException, handleValidationException etc. that return structured error responses with appropriate HTTP status codes. This avoids try-catch in every controller. You can also implement HandlerExceptionResolver or extend ResponseEntityExceptionHandler for deeper customisation.

Q11. What is the N+1 query problem and how do you fix it? Hard

If you have a list of 100 Rides and access ride.getDriver() on each, JPA fires 1 query for rides + 100 queries for drivers = 101 queries. Fix options: (1) JOIN FETCH in JPQL: SELECT r FROM Ride r JOIN FETCH r.driver. (2) @EntityGraph on repository method. (3) @BatchSize(size=50) on the association — fetches 50 drivers per query. (4) DTO projections with a single query. Always check generated SQL with show-sql=true or Hibernate statistics.

Q12. What is the difference between @PathVariable, @RequestParam, and @RequestBody? Easy

@PathVariable extracts values from the URI path (/rides/{id}). @RequestParam extracts query parameters (/rides?status=ACTIVE). @RequestBody deserialises the HTTP request body (JSON) into a Java object using Jackson — used for POST/PUT payloads. @RequestParam can have defaultValue and required=false. @RequestBody requires Content-Type: application/json header.

Q13. How do you write unit tests for a Spring Boot service? Medium

Use @ExtendWith(MockitoExtension.class) for pure unit tests — mock dependencies with @Mock, inject with @InjectMocks. For controller layer use @WebMvcTest (loads only web layer) with MockMvc. For integration tests use @SpringBootTest with @AutoConfigureMockMvc. Use @DataJpaTest for repository tests with an in-memory H2 DB. Mock external calls (Kafka, Stripe) using Mockito.when() or TestContainers for real infrastructure.

Q14. What are Spring profiles and how do you use them? Easy

Profiles allow environment-specific configuration. Define application-dev.yml, application-prod.yml. Activate with spring.profiles.active=dev (env var, JVM arg, or in application.yml). Use @Profile('dev') on @Bean or @Component to conditionally register beans. In Docker Compose, pass SPRING_PROFILES_ACTIVE=prod as an environment variable. This lets you use H2 in dev and PostgreSQL in prod without code changes.

Q15. Explain idempotency and why it matters in microservices. Hard

An operation is idempotent if calling it multiple times produces the same result as calling it once. Critical in microservices because networks are unreliable — a client may retry a request not knowing if the original succeeded. For payments, you must ensure retries don't double-charge. Implement by: (1) using a client-generated idempotency key, (2) storing processed request IDs in DB, (3) returning the same response for duplicate requests. Kafka consumer retries also require idempotent message handlers.

Section 3 — Apache Kafka Deep Dive

Q1. What is Kafka? How does it differ from RabbitMQ?

Medium

Kafka is a distributed, append-only, fault-tolerant event streaming platform. Unlike RabbitMQ (a message broker where messages are deleted after consumption), Kafka retains messages for a configurable period — consumers can replay from any offset. Kafka is partitioned for horizontal scaling and ordered within a partition. RabbitMQ supports more routing patterns (topic, fanout, direct) and is better for task queues. Kafka is better for event streaming, audit logs, and high-throughput scenarios.

Q2. What is a Kafka partition and why does it matter?

Medium

A topic is split into partitions — immutable, ordered logs of messages. Partitions enable parallelism: multiple consumer instances in a group can read from different partitions simultaneously. A consumer group with 3 instances and 3 partitions gives 3x parallel throughput. Messages with the same key always go to the same partition (preserving order for that key). More partitions = more parallelism but also more overhead on brokers and clients.

Q3. What is a consumer group? What happens when a consumer crashes?

Hard

A consumer group is a set of consumers that jointly consume a topic — each partition is assigned to exactly one consumer in the group. This gives load balancing. When a consumer crashes, Kafka detects a missed heartbeat and triggers a rebalance — Kafka redistributes that consumer's partitions among the surviving consumers. During rebalance, consumption pauses briefly. The new consumer resumes from the last committed offset, so no messages are lost.

Q4. What is the difference between at-least-once, at-most-once, and exactly-once delivery?

Hard

At-most-once: messages may be lost, never duplicated (auto-commit before processing). At-least-once: no loss but possible duplicates (commit after processing — on crash, message redelivered). Exactly-once: no loss, no duplicates — requires idempotent producer (enable.idempotence=true) + transactional API on producer AND idempotent consumers (dedup in DB). In our project we use at-least-once with idempotent consumers (DB check before payment processing).

Q5. What is the difference between Kafka offset commit strategies?

Hard

Auto-commit (enable.auto.commit=true): offsets committed periodically on a timer — simple but risky (may commit before processing completes). Manual sync commit (commitSync): blocks until broker acknowledges — safe but slow. Manual async commit (commitAsync): non-blocking, callback-based — fast but may fail silently. Best practice for critical consumers: disable auto-commit, process the message, then commitSync. For exactly-once: use Kafka transactions.

Q6. What is a Dead Letter Topic (DLT) in Kafka?

Medium

When a consumer fails to process a message after N retries, instead of blocking the partition indefinitely, the message is sent to a Dead Letter Topic (e.g., ride.completed.DLT). This allows normal messages to continue flowing while failed messages are quarantined for investigation and manual replay. Spring Kafka's @RetryableTopic annotation handles this automatically — configure backoff intervals and max retries, and it creates the DLT automatically.

Q7. How did you configure Kafka producer for reliability in your project? Hard

Three settings: (1) acks=all — leader waits for all in-sync replicas to acknowledge before confirming to producer, ensuring no data loss. (2) retries=3 — automatically retry transient send failures. (3) enable.idempotence=true — assigns a sequence number to each message so the broker deduplicates retries, preventing duplicate messages from retries. These three together give 'effectively exactly-once' producer semantics.

```
config.put(ACKS_CONFIG, "all");
config.put(RETRIES_CONFIG, 3);
config.put(ENABLE_IDEMPOTENCE_CONFIG, true);
```

Q8. What is Kafka Log Compaction? Hard

Log compaction retains only the latest message for each key in a topic, compacting away older versions. Useful for 'changelog' topics where you only care about current state (e.g., driver profile updates). Different from TTL-based retention which deletes old messages by time. After compaction, a new consumer starting from offset 0 gets the latest state for each key — useful for event sourcing and rebuilding state.

Q9. How do you monitor Kafka consumer lag? Medium

Consumer lag = (latest offset in partition) - (consumer's committed offset). High lag means the consumer is falling behind. Monitor via: (1) kafka-consumer-groups.sh --describe --group , (2) JMX metrics (records-lag-max per partition), (3) Prometheus + kafka-exporter scraping lag metrics, (4) Kafka UI (provectuslabs) which shows lag visually. Alert when lag exceeds a threshold — it usually means the consumer is slow or crashed.

Q10. What is Kafka Streams vs plain Kafka consumer? Hard

Plain Kafka consumer: you poll messages, process, commit offsets — good for simple event-driven flows. Kafka Streams: a library for stateful stream processing — windowed aggregations, joins between topics, real-time analytics. It handles state stores (backed by RocksDB + changelog topics), fault tolerance, and exactly-once semantics transparently. Use plain consumer for simple event routing; use Kafka Streams for complex aggregations like 'count rides per driver per hour'.

Q11. How do you ensure message ordering in Kafka? Medium

Ordering is guaranteed within a partition but not across partitions. To ensure ordered processing for a given entity (e.g., all events for rideId='123'), always produce with the same key (rideId) — Kafka routes same-key messages to the same partition. On the consumer side, a single thread processes each partition sequentially. If you use concurrency >1 in ConcurrentKafkaListenerContainerFactory, each thread handles different partitions, so ordering per key is still preserved.

Q12. What is the role of ZooKeeper in Kafka? Is it still required? Medium

Historically, ZooKeeper managed Kafka broker metadata, leader election, and consumer group coordination. From Kafka 2.8+ (KRaft mode), Kafka has an internal Raft-based consensus mechanism and no longer requires ZooKeeper. Kafka 3.x made KRaft production-ready. In our project we still use ZooKeeper with Confluent's Kafka image for stability, but new projects should use KRaft mode to simplify operations and reduce infrastructure.

Section 4 — Redis Deep Dive

Q1. What data structures does Redis support? Give use cases. Medium

String: counters, cached values, JWT blacklist. List: message queues, activity feeds. Set: unique visitors, tags. Sorted Set: leaderboards (score-based ranking), rate limiting with sliding window. Hash: storing objects (user profile fields). GeoSet: geo-indexed keys for location queries (GEORADIUS). HyperLogLog: approximate unique count. Stream: append-only log (similar to Kafka for simpler use cases). In our project: GeoSet for driver locations, String for surge multipliers and ride TTLs.

Q2. How does Redis handle persistence? Hard

Two mechanisms: (1) RDB (Redis Database) — periodic point-in-time snapshots to disk. Fast restarts, larger data loss window. Configure with save 900 1 (snapshot if 1 key changed in 900s). (2) AOF (Append-Only File) — logs every write operation. Durability options: always (sync on every write), everysec (sync every second), no (OS decides). everysec balances durability and performance. For production, use both RDB + AOF. In our project, Redis is used as a cache so we tolerate some data loss.

Q3. What is Redis Cache eviction? What policies exist? Medium

When Redis hits maxmemory limit, eviction kicks in based on policy: noeviction (return error), allkeys-lru (evict least recently used across all keys), volatile-lru (LRU only among keys with TTL), allkeys-lfu (least frequently used), volatile-ttl (evict soonest-to-expire keys), allkeys-random. For a cache, allkeys-lru or allkeys-lfu is typical. In our docker-compose.yml we set maxmemory 256mb and maxmemory-policy allkeys-lru.

Q4. How does Redis GEORADIUS work internally? Hard

Redis uses Geohash encoding — latitude/longitude pairs are encoded into a 52-bit integer and stored in a Sorted Set (ZSET) using the geohash as the score. GEORADIUS converts the search centre + radius to a set of geohash prefixes covering the area, then queries the ZSET for members whose scores fall within those ranges. This is O(N+log M) where N is results and M is total members — very efficient for nearby queries.

Q5. What is Redis Sentinel vs Redis Cluster? Hard

Sentinel: high availability for a single shard — monitors master + replicas, performs automatic failover when master dies, notifies clients of new master. No horizontal scaling (data fits on one node). Cluster: shards data across 16384 hash slots spread across multiple master nodes, each with replicas. Provides both HA and horizontal scaling. Use Sentinel for simpler setups where data fits in one node; use Cluster when data exceeds single-node capacity.

Q6. What is the difference between EXPIRE and EXPIREAT in Redis? Easy

EXPIRE key seconds — sets a relative TTL from now. EXPIREAT key timestamp — sets an absolute Unix timestamp for expiry. PEXPIRE/PEXPIREAT work in milliseconds. In our project, ride:request:{id} uses EXPIRE with 60s TTL so it auto-expires 60 seconds after creation regardless of when it was accessed. Use EXPIREAT when you need to expire at a specific wall-clock time (e.g., end of business day).

Q7. How do you implement distributed rate limiting with Redis? Hard

Using a sliding window counter with Redis: INCR rate:{userId}:{windowStart} increments the counter for the current time window. EXPIRE sets the TTL to the window size. If counter > limit, reject. For a more precise sliding window, use a Sorted Set: ZADD rate:{userId} timestamp requestId, ZREMRANGEBYSCORE to remove old entries, ZCARD to count. In our API Gateway, Spring Cloud Gateway's RequestRateLimiter filter uses Redis with a token bucket algorithm via RedisRateLimiter.

Q8. What is Redis Pub/Sub? How does it differ from Kafka? Medium

Redis Pub/Sub is a fire-and-forget messaging system. Publishers send messages to channels; subscribers receive them in real-time. Key difference from Kafka: (1) No persistence — if a subscriber is offline, messages are lost. (2) No consumer groups — all subscribers get every message. (3) No offset replay. Redis Pub/Sub is good for real-time notifications (e.g., chat, live score updates) where losing a message is acceptable. Kafka is for durable, replayable event streams.

Q9. How do you handle cache stampede (thundering herd) in Redis? Hard

Cache stampede happens when a cached value expires and multiple requests simultaneously find a cache miss and all hit the DB. Solutions: (1) Mutex lock — first request acquires a Redis lock (SET key NX EX), fetches from DB and populates cache; others wait. (2) Probabilistic early expiration — randomly refresh the cache before it expires (based on beta). (3) Background refresh — a separate thread proactively refreshes caches before TTL expires. (4) Stagger TTLs with jitter to prevent mass simultaneous expiry.

Q10. What is the difference between Redis String SET and SETNX? Medium

SET key value sets unconditionally. SETNX (SET if Not eXists) only sets if the key doesn't exist — returns 1 on success, 0 if key already existed. SETNX is used for distributed locking: SET lock:resource uniqueValue NX EX 30 — atomically sets only if no lock exists, with a TTL. This is the foundation of the Redlock algorithm. Modern Redis uses SET key value NX EX seconds as a single atomic command instead of separate SETNX + EXPIRE.

Section 5 — System Design

Q1. What is the CAP theorem? Which did your system choose?

Hard

CAP: in a distributed system you can guarantee only 2 of 3: Consistency (every read sees the latest write), Availability (every request gets a response), Partition Tolerance (system works despite network partitions). Partition tolerance is non-negotiable in distributed systems, so the real choice is CP vs AP. Our system is mostly AP — we prefer availability over strict consistency. Redis may serve slightly stale surge data; Kafka consumers may process events slightly delayed. For payments, we lean CP — we use DB transactions and idempotency keys.

Q2. What is eventual consistency? Where does it appear in your project?

Hard

Eventual consistency means that given no new updates, all replicas will eventually converge to the same value. In our project: (1) Driver location in Redis may lag a few seconds behind actual GPS position — eventually consistent. (2) Ride status in Driver Service's view lags behind Ride Service until Kafka event is consumed. (3) Payment status is updated asynchronously via Kafka — the ride may show COMPLETED while payment is still PENDING for a few seconds. This is acceptable for our use case.

Q3. How would you design a rate limiter for the API?

Hard

Options: (1) Fixed window — count requests per user per time window in Redis. Simple but allows burst at window boundary. (2) Sliding window log — store each request timestamp in a Redis Sorted Set, count entries in the last N seconds. Accurate but memory-intensive. (3) Token bucket — add tokens at a fixed rate, consume on each request. Allows bursts up to bucket size. (4) Leaky bucket — requests processed at constant rate, excess queued or dropped. Token bucket is most common in practice (used by Spring Cloud Gateway).

Q4. How would you design the notification system at scale?

Hard

For millions of users: (1) Kafka as the event bus with notification-service as consumer. (2) Fan-out: for events with many recipients, use parallel processing with multiple consumer threads. (3) Push notifications via Firebase FCM (batched API — up to 500 tokens per request). (4) SMS via Twilio with retry on failure. (5) Priority queues — critical notifications (payment failed) on a high-priority Kafka topic with dedicated consumers. (6) User preferences: check Redis cache for notification opt-out before sending.

Q5. How would you handle database schema migrations in microservices?

Medium

Use Flyway or Liquibase per service. Each service owns its migration scripts in src/main/resources/db/migration. On startup, Flyway compares current DB version with migration scripts and applies missing ones. Migrations must be backward-compatible during rolling deploys (add columns don't remove them; make new columns nullable before populating). For zero-downtime: deploy new code first (tolerates missing column), run migration, then remove old code.

Q6. What is CQRS and would it apply to your project?

Hard

CQRS (Command Query Responsibility Segregation) separates write (command) and read (query) models. Commands mutate state; queries return data from an optimised read model. In our project it

could apply to the Ride history feature: writes go to PostgreSQL (normalised), but read queries (rider's trip history with driver details, fare, timestamps) could be served from a denormalised Elasticsearch or MongoDB read model, updated via Kafka events. This avoids expensive JOINs on the write DB.

Q7. How would you scale the ride-matching to handle 10 million active drivers? Hard

Redis Cluster with geo-sharding by city (drivers:locations:mumbai, drivers:locations:delhi). Each city's geo data fits on one shard. For matching, a Geo Matching Service queries the relevant city shard. For global events, use a message fan-out layer. Add read replicas for GEORADIUS queries. Use consistent hashing to add/remove Redis nodes without full rehashing. Also consider H3 geospatial indexing for hierarchical region queries at different resolutions.

Q8. What is the difference between horizontal and vertical scaling? Easy

Vertical scaling (scale up): add more CPU/RAM to existing servers. Simple but has physical limits and causes downtime. Horizontal scaling (scale out): add more server instances behind a load balancer. Stateless services scale horizontally easily. Stateful services (DBs) need sharding/replication. In our project, all microservices are stateless (state in Redis/Postgres) and can be horizontally scaled by running multiple instances — Eureka load-balances across them automatically.

Q9. How would you implement a global search feature across microservices? Hard

Use Elasticsearch as a dedicated search service. Each microservice publishes change events to Kafka (ride.completed, driver.updated). A Search Indexer service consumes these events and updates Elasticsearch documents. The Search Service exposes a REST API that queries Elasticsearch for full-text search across rides, drivers, and users. This avoids cross-service queries and leverages Elasticsearch's inverted index for fast full-text search, geo queries, and aggregations.

Q10. How do you handle service-to-service authentication? Hard

Options: (1) Mutual TLS (mTLS) — each service has a certificate; services verify each other's identity. Most secure. (2) JWT service tokens — services carry a service-level JWT signed with a shared secret; downstream services verify it. (3) API keys per service stored in Config Server. In a Kubernetes environment, use a service mesh like Istio which handles mTLS transparently without application code changes. In our project, X-User-Id headers are trusted because they only come from the API Gateway — internal services are in a private Docker network.

Section 6 — Managerial Round (Project-Based)

Q1. Tell me about this project. What is it and why did you build it? Medium

This is a production-grade ride-sharing backend similar to Uber/Ola, built to demonstrate real-world microservices architecture. I chose this because it naturally involves all the hard problems in distributed systems: real-time location tracking, event-driven workflows, payment processing, and surge pricing. I wanted to show not just that I can write Spring Boot code, but that I understand how systems fail and how to design for resilience and scale. Every architectural decision — Kafka for decoupling, Redis for geo queries, Resilience4j for circuit breaking — has a specific reason behind it.

Q2. What was the most challenging part of building this project? Hard

The most challenging part was getting the payment flow right. It's easy to charge a user; it's hard to ensure you never charge them twice and never miss a charge. I had to design two independent layers of idempotency — a DB-level check and a Stripe idempotency key — because Kafka can redeliver messages. I also had to think about what happens when Stripe is down mid-transaction: the circuit breaker opens, the payment is queued for retry, and the rider is notified gracefully instead of seeing a cryptic error. Getting this flow correct required understanding the Saga pattern deeply.

Q3. If you had to add a new feature — 'scheduled rides' — how would you approach it? Hard

I'd add a Scheduled Ride entity with a scheduledTime field. A new Scheduler Service would use a cron job or Redis sorted set (score = scheduledTime as Unix timestamp) to find rides due in the next N minutes and publish ride.requested events to Kafka at the right time. The rest of the flow reuses existing services unchanged. I'd also add a scheduled_ride.created Kafka topic for audit purposes. This shows how event-driven architecture makes adding features low-risk — no existing service changes.

Q4. How would you handle a production incident where payments are failing? Hard

Step 1: Check Kafka consumer lag on payment.failure topic — is it growing? Step 2: Check Grafana for Payment Service error rate spike and Stripe API response times. Step 3: Check Zipkin traces for the failing payment flow — where is it failing? Step 4: Check Resilience4j circuit breaker state — is it open? Step 5: If Stripe is down, the circuit breaker should have opened and no new charges are attempted. Communicate status to users. Step 6: Once Stripe recovers, PAYMENT_PENDING rides are retried. Step 7: Post-incident review — add better alerting on Kafka consumer lag for payment topics.

Q5. How did you approach testing this system? Medium

Three levels: (1) Unit tests — each service method tested with Mockito, mocking Kafka, Redis, and repositories. Fast feedback loop. (2) Integration tests — @SpringBootTest with TestContainers (real PostgreSQL, Redis containers) for testing DB interactions and Kafka flows end-to-end within a service. (3) Contract tests — Pact or Spring Cloud Contract to verify API Gateway routes match service expectations without spinning up the full system. For manual testing, docker-compose up gives a full local environment with Kafka UI to inspect events.

Q6. If you were to improve this project, what would you add next? Hard

Four priorities: (1) Kubernetes deployment with Helm charts — replace docker-compose for production-grade orchestration, auto-scaling, and rolling deploys. (2) API versioning strategy —

currently all routes are /api/v1/... implicitly; explicit versioning prevents breaking clients. (3) Distributed caching invalidation strategy — currently caches have TTLs; adding event-driven invalidation would reduce stale data windows. (4) gRPC for internal service communication — faster than JSON REST for high-frequency internal calls like location updates.

Q7. How do you ensure your code is maintainable over time? Medium

Several practices: (1) Each service has a single responsibility — new features rarely require changing multiple services. (2) Events are the contract between services — changing internal DB schema doesn't affect other services. (3) Centralised config in Config Server means no config scattered across codebases. (4) Consistent package structure across services (controller/service/repository/model/dto/event) makes it easy for any developer to navigate. (5) Meaningful Kafka topic names (ride.requested, not just 'events') serve as living documentation.

Q8. How would you explain this architecture to a non-technical stakeholder? Easy

Imagine our platform as a city's delivery network. The API Gateway is the front desk — everyone enters through it and it checks IDs (JWT). Each department (User, Driver, Ride, Payment) is independent and communicates via a message board (Kafka) — no department calls another directly. The notice board is always available even if a department is temporarily closed. Redis is our real-time map — it always shows where every driver is. When a payment fails, the Accounts department (Payment Service) queues it for retry rather than losing it.

Q9. How did you prioritise which features to build in this project? Medium

I focused on the core happy path first: user registers, driver goes online, rider requests ride, driver accepts, ride completes, payment succeeds. Once that end-to-end flow worked with Kafka and Redis, I added resilience: circuit breakers, TTL-based ride expiry, idempotent payments. Then observability: Prometheus metrics, Zipkin tracing. Finally, I added surge pricing as an advanced feature. This mirrors how real teams ship — working core first, then edge cases, then enhancements.

Q10. Describe a design decision you made and the trade-off involved. Hard

I chose choreography-based Saga over orchestration for the payment flow. Choreography: services react to events independently — simpler, no single point of failure. Orchestration: a central saga orchestrator controls the flow — easier to visualise but adds a component that can fail. I chose choreography because it keeps services decoupled and there's no additional infrastructure. The trade-off: the flow is harder to trace in code — you have to follow Kafka topics to understand the sequence. I mitigated this with Zipkin distributed tracing and clear event naming conventions.

Q11. How would you onboard a new developer to this codebase? Medium

Three steps: (1) Read the architecture document — understand the 7 services, Kafka topics, and Redis usage before touching code. (2) Run docker-compose up and go through a full ride flow via Postman — see Kafka events in Kafka UI, watch logs in real time. (3) Start with the simplest service (User Service) to understand the package structure, then move to Ride Service to understand Kafka production, then Payment Service for Kafka consumption and circuit breakers. The consistent structure across all services means patterns learned in one apply everywhere.

Q12. What would you do differently if you started this project over? Hard

Three things: (1) Add Kubernetes manifests from day one instead of docker-compose — production deployments need resource limits, health probes, and auto-scaling from the start. (2) Define event schemas with Avro + Schema Registry — currently events are plain JSON with no schema enforcement; a schema change could silently break consumers. (3) Use @Transactional outbox pattern for Kafka publishing — currently there's a rare race condition where the DB transaction commits but Kafka publish fails. The outbox pattern guarantees at-least-once Kafka delivery atomically with DB writes.

Good luck with your interviews!

Remember: interviewers want to see that you understand WHY, not just HOW.